

第七章 分区、文件系统、Swap 与持久挂载

存储题的危险不在命令复杂，而在设备名相似且写操作不可逆。磁盘存在不代表空闲，分区存在不代表有文件系统，目录存在不代表已挂载，当前 Swap 可用也不代表重启后启用。本章从设备证据开始，沿分区、签名、文件系统、挂载与持久配置逐层验收。

[概念] 块设备可承载分区表、分区、LVM 元数据、文件系统或 Swap 签名。GPT 与 MBR 描述分区布局；文件系统在块设备上组织文件；挂载把文件系统连接到统一目录树。

[概念] UUID 属于文件系统或 Swap 签名，不是磁盘永久属性。重新格式化会生成新 UUID；`/etc/fstab` 使用旧 UUID 时，配置与当前对象会脱节。

[操作语义] `lsblk` 展示块设备拓扑，`blkid` 读取签名，`wipefs` 默认列出现有签名；`parted` / `fdisk` 修改分区表，`partprobe` 请求内核重新读取分区。

[操作语义] `mkfs.xfs` / `mkfs.ext4` 新建文件系统，`mount` 建立当前挂载，`findmnt` 查询挂载关系；`mkswap` 创建 Swap 签名，`swapon` / `swapoff` 改变当前启用状态。

[知识专题] 设备、分区、签名、文件系统与挂载层级

① [知识点] 六层状态不能互相替代

典型链路是“磁盘 → 分区 → 文件系统/Swap 签名 → 当前挂载或 `swapon` → `fstab` 持久条目”。每层都有独立查询：

```
lsblk -o NAME,PATH,SIZE,TYPE,FSTYPE,UUID,MOUNTPOINTS
blkid
findmnt
swapon --show
```

`lsblk` 可把多层信息放在一张表中，但仍需按列解释。FSTYPE 为空不充分证明安全空闲，设备可能包含未识别数据或分区表；写操作前还要核对已有挂载、LVM/RAID 归属和题意。

② [知识点] GPT 与 MBR 的考试边界

GPT 支持更多分区和大容量设备，使用 GUID 类型；MBR 具有主/扩展/逻辑分区等旧限制。RHEL 9 新磁盘通常优先 GPT，除非题目指定 MBR 兼容需求。选择分区表会重写磁盘布局，不是在已有数据盘上随意切换的显示选项。

③ [知识点] XFS 与 ext4 的增长/缩小边界

XFS 支持在线增长但不支持原地缩小；ext4 可增长，并在卸载等条件下缩小。创建文件系统的 `mkfs` 会重建结构，不是修复或扩容命令。题目只要求创建时按指定类型执行，不因个人偏好替换。

④ [验证点] 设备身份必须来自多条一致证据

用设备 PATH、容量、拓扑、型号/序列（可用时）、签名、挂载和上层归属共同判断。`/dev/sdb`、`/dev/vdb` 等枚举名可能随环境变化；操作前重新运行查询，不能依赖上一次考试练习中的设备名。

[Cheatsheet] 拓扑 `lsblk`；签名 `blkid/wipefs`；当前挂载 `findmnt`；Swap `swapon --show`；磁盘存在 $\neq$ 空闲；`mkfs/mkswap` 都是写签名操作。

[操作专题] 在确认空闲的磁盘上创建 GPT 分区

① [操作点] 写入前建立设备基线

```
lsblk -o NAME,PATH,SIZE,TYPE,FSTYPE,UUID,MOUNTPOINTS
blkid /dev/vdb /dev/vdb1
wipefs /dev/vdb # 只列签名，不加 -a
pvs; vgs; lvs # 排除 LVM 归属
```

若发现文件系统、分区、挂载或现有 PV，停止并核对题意。`wipefs -a` 会擦除签名，不是调查命令。

② [参数点] parted 的单位和边界必须明确

`parted -s` 非交互执行，`mklabel gpt` 创建 GPT，`mkpart` 指定名称/类型和起止位置。使用 MiB/GiB 避免十进制与二进制含糊，并预留开头对齐空间。

```
parted -s /dev/vdb mklabel gpt # 新建 GPT
parted -s /dev/vdb mkpart data xfs 1MiB 2049MiB # 创建约 2 GiB 分区
parted -s /dev/vdb print
partprobe /dev/vdb # 请求内核重读
udevadm settle # 等待设备节点事件
```

`parted` 中的 `xfs` 参数主要设置预期/类型提示，不会创建 XFS；文件系统仍需独立 `mkfs.xfs`。

③ [验证点] 分区表与内核设备视图一致

```
parted /dev/vdb unit MiB print
lsblk -o NAME,START,SIZE,TYPE,FSTYPE /dev/vdb
```

如果新分区节点未出现，先检查内核是否仍在用旧布局和命令错误，不要重复 `mklabel`。生产环境可能需要重启才能安全重读被占用磁盘，但考试新盘应先确认没有使用者。

[Cheatsheet] 查询签名/归属 $\rightarrow$ `mklabel gpt` $\rightarrow$ `mkpart ... 1MiB <END>` $\rightarrow$ `partprobe / udevadm settle` $\rightarrow$ `parted print` + `lsblk`；`mkpart` 不创建文件系统。

[操作专题] 创建文件系统并用 UUID 持久挂载

① [操作点] 只在新分区上创建指定文件系统

`mkfs.xfs` 与 `mkfs.ext4` 会覆盖现有文件系统结构。确认 `/dev/vdb1` 是新建空分区后执行：

```
mkfs.xfs -L DATA /dev/vdb1          # 新建 XFS 并设置 label
mkdir -p /srv/data
blkid /dev/vdb1                      # 获取真实 UUID、TYPE、LABEL
```

label 可读但不一定全局唯一；UUID 更适合 `fstab`。UUID 必须从当前对象查询，不能复制示例。

② [知识点] `fstab` 六字段分别表达对象和策略

六字段是设备标识、挂载点、类型、选项、dump、fsck 顺序。XFS 常用末两项 `0 0`；ext4 根外文件系统常见 fsck pass 为 2，但应服从题意和系统惯例。

```
UUID=<真实 UUID> /srv/data xfs defaults 0 0
```

`nofail` 会改变启动失败处理，不应为了掩盖错误 UUID 默认添加。挂载选项如 `ro`、`noexec`、`nodev`、`nosuid` 会改变功能边界，只有目标明确时设置。

③ [操作点] 在重启前验证配置并实际挂载

```
systemctl daemon-reload          # 重读 fstab 生成的 mount units
findmnt --verify                  # 检查明显语法/引用问题
mount -a                          # 实际尝试未挂载条目
```

`findmnt --verify` 不等于实际挂载，`mount -a` 无报错也不证明挂载源就是目标分区。继续查询目标目录。

④ [验证点] 当前、容量、UUID 与写入功能

```
findmnt /srv/data
df -hT /srv/data
lsblk -f /dev/vdb
touch /srv/data/.write-test && rm /srv/data/.write-test
```

`findmnt` 证明挂载源/类型/选项，`df` 证明文件系统视角容量，`lsblk -f` 交叉检查 UUID。写入测试还受目录权限和只读选项影响。

[Cheatsheet] `mkfs.<TYPE>` → `blkid` 取 UUID → 写 `fstab` → `daemon-reload` → `findmnt --verify` → `mount -a` → `findmnt` / `df` / `lsblk` / 功能测试。

[操作专题] 创建并持久启用 Swap

① [知识点] Swap 签名、当前启用和持久条目是三层

分区创建后，`mkswap` 写入 Swap 签名并产生 UUID；`swapon` 加入当前 Swap；`fstab` 决定启动时启用。任一层成功不代表其他层完成。

② [操作点] 创建 Swap 分区并启用

假设 `/dev/vdb2` 已由分区步骤创建且确认空闲：

```
mkswap -L EXAMSWAP /dev/vdb2      # 创建 Swap 签名
blkid /dev/vdb2                   # 获取真实 UUID
swapon /dev/vdb2                  # 当前启用
```

`fstab` 条目：

```
UUID=<真实 UUID> none swap defaults 0 0
```

③ [验证点] 签名、当前状态和持久配置

```
swapon --show --output=NAME,TYPE,SIZE,USED,PRI0
cat /proc/swaps
findmnt --verify
```

`free -h` 显示 Swap 总量和使用量，但不直接说明由哪个设备提供；`swapon --show` 更适合设备验收。Swap 未被使用不表示未启用。

④ [边界] 优先级与删除顺序

多个 Swap 可设 `pri=`，数值较高者优先。删除 Swap 前先 `swapoff` 并确认内存能够容纳迁移，再移除 `fstab` 和签名/分区；这不属于创建题的通用清理流程。

[Cheatsheet] `mkswap` 写签名 → `blkid` → `swapon` 当前启用 → `fstab none swap defaults 0 0` → `swapon --show` + `findmnt --verify`；`USED=0` 仍可正常启用。

[知识专题] mount 选项、systemd units 与卸载边界

① [知识点] fstab 会被 systemd 转换为 mount unit

路径 `/srv/data` 对应 unit 名通常为 `srv-data.mount`，可用 `systemd-escape --path --suffix=mount /srv/data` 计算。`systemctl status srv-data.mount` 和 `journal` 可提供启动挂载失败证据。

② [知识点] 卸载失败说明对象仍被使用

`umount /srv/data` 报 `target is busy` 时，用 `findmnt` 确认嵌套挂载，用 `fuser -vm` 或 `lsof` 查打开文件与工作目录。不要先使用 `lazy/force` 卸载；应让进程释放对象并离开目录。

③ [边界] 挂载会遮蔽目录原有内容

挂载到非空目录后，原目录内容被挂载文件系统遮蔽但没有删除。卸载后会重新出现。题目要求保留数据时，在挂载前检查目录是否为空；不要把“挂载后文件消失”误判为数据被删除。

[Cheatsheet] `mount unit` 用 `systemd-escape` 计算；`busy` 查嵌套挂载和 `fuser -vm`；挂载遮蔽原目录内容；不以 `force/lazy` 作为第一修复。

[诊断专题] 根据症状定位分区、签名、挂载或持久层

① [诊断点] 目录存在但数据写入根文件系统

`findmnt /srv/data` 无结果表示当前未挂载；检查 `fstab`、`findmnt --verify`、`mount -a` 和 `mount unit journal`。不要重新 `mkfs`。

② [诊断点] `fstab` 报 `UUID` 不存在

用 `blkid / lsblk -f` 找当前 `UUID`，检查是否复制错误或重新格式化后 `UUID` 已变。修正条目后重新 `verify/mount`；不要再格式化以“得到新 `UUID`”。

③ [诊断点] 分区已创建但内核看不到

核对 `parted print` 是否真的写入，运行 `partprobe / udevadm settle`，检查设备是否被使用导致重读失败。不要重复创建同一分区。

④ [诊断点] `Swap` 签名存在但总量未增加

`blkid` 只证明签名；检查 `swapon --show` 和 `swapon <DEV>` 的错误，再检查 `fstab` 持久层。

[Cheatsheet] 目录无挂载查 `fstab/mount unit`；`UUID` 错查当前签名；分区节点缺失查内核重读；`Swap` 不见查 `swapon --show`；任何层失败都不重建上层对象。

[经典任务] 在新磁盘上创建 XFS 与 Swap 并持久启用

服务器提供一块确认用于本题但当前状态未知的 `/dev/vdb`。在确认没有需要保留的数据后，使用 GPT 创建约 2 GiB 的分区用于 XFS，持久挂载到 `/srv/data`；再创建约 1 GiB 的 Swap 分区并当前启用、重启后保持。

XFS 挂载必须使用真实 UUID，目录原有内容如存在必须先报告并保留。不得使用 `wipefs -a`、强制格式化或重启代替调查。验收包括分区边界、签名、UUID、当前挂载、文件系统容量、写入功能、Swap 当前设备和 `fstab` 静态检查。

请先独立完成。参考解答从下一页开始。

[参考解答] 从设备证据到两类持久状态

① [操作点] 调查并创建分区

```
lsblk -o NAME,PATH,SIZE,TYPE,FSTYPE,UUID,MOUNTPOINTS
blkid /dev/vdb /dev/vdb1 /dev/vdb2
wipefs /dev/vdb
pvs
parted -s /dev/vdb mklabel gpt
parted -s /dev/vdb mkpart data xfs 1MiB 2049MiB
parted -s /dev/vdb mkpart swap linux-swap 2049MiB 3073MiB
partprobe /dev/vdb; udevadm settle
parted /dev/vdb unit MiB print
```

只有调查确认可写后才执行 mklabel。

② [操作点] 创建 XFS 与 Swap 签名

```
test ! -e /srv/data || find /srv/data -mindepth 1 -maxdepth 1 -print
mkfs.xfs -L DATA /dev/vdb1
mkswap -L EXAMSWAP /dev/vdb2
mkdir -p /srv/data
blkid /dev/vdb1 /dev/vdb2
```

将两个真实 UUID 写入 fstab 的 XFS 和 Swap 条目。

③ [操作点] 当前启用并验证持久配置

```
systemctl daemon-reload
findmnt --verify
mount -a
swapon -a
```

④ [验证点] 分层验收

```
lsblk -f /dev/vdb
findmnt /srv/data
df -hT /srv/data
touch /srv/data/.check && rm /srv/data/.check
swapon --show --output=NAME,TYPE,SIZE,USED,PRI0
findmnt --verify
```

[Cheatsheet] 设备/签名/归属调查 → GPT 两分区 → 内核重读 → `mkfs.xfs` / `mkswap` → 双 UUID 写 fstab → `mount -a` / `swapon -a` → 文件系统与 Swap 分层验收。

[本章收束] 每个写操作只推进一个存储层

分区工具改变布局，mkfs/mkswap 写签名，mount/swap on 改变当前状态，fstab 描述启动持久性。目录、分区或 UUID 的存在都只能证明自己的层，不能自动代表应用可用。

稳定路径始终从设备身份和已有数据开始，写入后立即用对应查询确认。错误 UUID、未重读分区表或未启用 Swap 都有局部修复入口，不需要重新格式化。把破坏性命令留在证据充分的创建阶段，存储题才能既完成目标又保留数据边界。